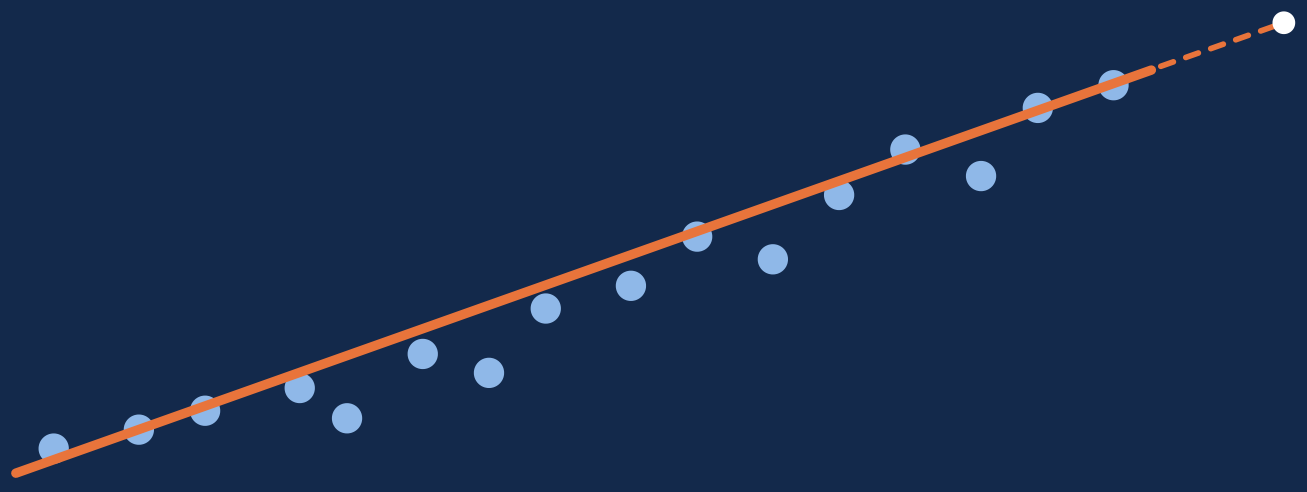
A FRIENDLY GUIDE TO MACHINE LEARNING

Predicting Exam Scores

How a computer learned the link between study hours and test scores, built from scratch with gradient descent, explained step by step.



BEFORE WE START

What's inside

1	What is linear regression?	*Drawing the best straight line*
2	The mission	*Framing the problem*
3	Meet the data	*25 students*
4	Detective work	*Is the pattern really a line?*
5	The model	$y = mx + b$
6	Gradient descent	*Walking downhill to the answer*
7	The learning rate	*Step size and convergence*
8	How good is the model?	MSE, RMSE, MAE, R^2, *residuals*
9	Checking our answer	*Exact formula and a fair test*
10	Predictions and limits	*Using it honestly*
11	Interview toolkit	*Pitch, Q&A, glossary*

How to use this book

Each chapter covers one step of the project, in the order you did it in the notebook. Every chapter has the same shape: the big idea, how it works, a picture, and a short remember box. Look out for these coloured boxes:

💡 **IN PLAIN ENGLISH**
The idea with zero jargon.

🎤 **INTERVIEW TIP**
How to say it when someone asks.

🗣️ **EXPLAIN IT TO AN AUDIENCE**
A one-liner or analogy anyone can follow.

⚠️ **WATCH OUT**
A common mistake to avoid.

The whole journey on one page

1. **Frame the problem**
   Predict a score from hours studied. Decide how to measure "good".
2. **Get the data**
   Load `score.csv`: 25 students, 2 columns.
3. **Explore**
   Scatter plot and correlation (0.976). It really is a line.
4. **Choose the model**
   A straight line: score = m × hours + b.
5. **Train it**
   Gradient descent: 1,000 small downhill steps to find m and b.
6. **Tune it**
   Pick a learning rate that is fast but doesn't explode.
7. **Evaluate**
   MSE, RMSE, MAE, R^2, residuals, and a train/test split.
8. **Use it**
   Predict new students' scores, and know the limits.

The project in four numbers

9.78
points gained per extra hour of study (the slope)

5.37
RMSE: typical error, in score points

0.953
R^2: hours explain 95% of the variation in scores

92.9
predicted score for a student who studies 9.25 hours

🎤 **INTERVIEW TIP**
What makes this project stand out is that you **didn't just call a library**. You wrote gradient descent yourself, then proved it was right by comparing it with the exact formula and scikit-learn. Lead with that.

CHAPTER 1

What is linear regression?

Finding the straight line that best fits the data

Normally a programmer writes the rules: "every hour of study adds 10 points." With machine learning we don't write the rule. We show the computer examples with the right answers, and it **works out the rule by itself**.

Linear regression is the simplest version of this. It assumes the output changes in a straight line as the input changes, and it finds the line that sits as close as possible to all the data points.

> 🗣️ EXPLAIN IT TO AN AUDIENCE
> Imagine pinning a ruler onto a scatter of dots so that it passes as close to all of them as possible. Linear regression is a precise way of finding where to put the ruler.

Three words you'll hear all the time

Word	Meaning	In our project
Feature (input, x)	The information we give the model (the clue)	Hours studied
Label (target, y)	The answer we want it to predict	Exam score (%)
Model	The rule the computer builds from the examples	score = 9.78 × hours + 2.46

What kind of problem is ours?

- **Supervised learning:** every example comes with the correct answer (the real score), like practising with an answer key.
- **Regression:** we predict a number. Predicting a category such as "pass" or "fail" would be *classification*.
- **Simple** linear regression: only **one** feature. With several features (hours, sleep, attendance) it would be *multiple* linear regression.

> 🎤 INTERVIEW TIP
> If asked "what type of ML problem was this?", say: **"A supervised, simple linear regression problem: supervised because each student has a known score, regression because the output is a number, and simple because there's a single input feature, hours studied."**

REMEMBER

- Linear regression = find the straight line that best fits the data.
- Our project: hours studied (feature) → model → exam score (label).

CHAPTER 2

The mission

Every good project starts with a clear question

A school wants to know: **if a student studies for a certain number of hours, what score can they expect?** It could use the answer to spot students who need support early, or to show students how much study time pays off.

> 💡 **IN PLAIN ENGLISH**
> Before writing code, ask: **What exactly are we predicting? Who will use it? What does "good" look like?** Here: predict a score (0 to 100) from hours, and "good" means a small typical error.

How do we measure "good"?

We need numbers that say how wrong the model is. For each student the **error** (or residual) is *real score − predicted score*. We then combine all the errors in three ways:

Metric	How it's calculated	What it tells you
MSE Mean Squared Error	Square each error, then average	The number gradient descent minimises. In "points squared", so hard to read directly.
RMSE Root MSE	Square root of the MSE	Back in score points: "typically off by about X points". Punishes big misses.
MAE Mean Absolute Error	Average of the errors, ignoring the sign	Also in points. Treats every miss equally.

> 🎯 **QUICK EXAMPLE**
> Three students with errors of 2, 4 and 9 points.
> **MAE** = (2 + 4 + 9) ÷ 3 = **5.0**. **MSE** = (4 + 16 + 81) ÷ 3 = **33.7**. **RMSE** = √33.7 = **5.8**.
> RMSE is higher than MAE because the one big miss (9) counts extra after squaring.

> 🎤 **INTERVIEW TIP**
> **Why square the errors?** Squaring makes every error positive (so +5 and −5 don't cancel out), punishes big mistakes more, and gives a smooth, bowl-shaped error surface that gradient descent can slide down. Absolute values have a sharp corner at zero, which is harder to optimise.

REMEMBER

- Goal: predict a student's score from the hours they study.
- Scores: MSE (what we minimise), RMSE and MAE (what we report). Lower is better.

CHAPTER 3

Meet the data

25 students, two columns, no missing values

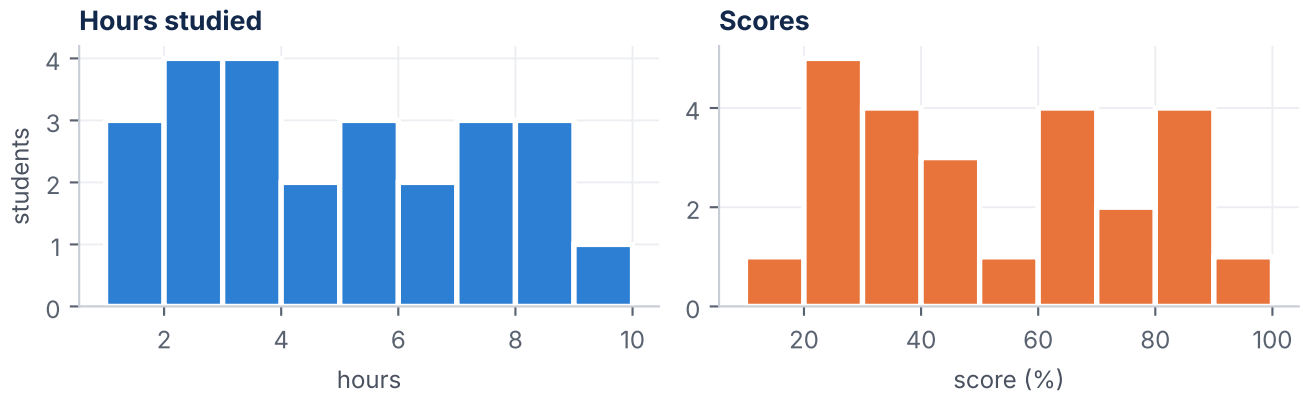
The dataset, `score.csv`, is small and clean: each row is one student.

Column	What it tells us	Min	Max	Mean	Std dev
`Hours`	Hours the student studied	1.1	9.2	5.01	2.53
`Scores`	Exam score in percent (the answer to predict)	17	95	51.48	25.29

```
data = pd.read_csv('score.csv')
display(data)                       # look at every row
display(data.describe())            # min, max, mean, std in one table
print(data.isnull().sum())          # check for missing values: 0 and 0
```

First look: histograms

A histogram shows how many students fall into each range.



Study hours are spread fairly evenly from 1 to 9. Scores cluster in the 20s and 30s, with a second group in the 60s to 90s.

⚠️ **WATCH OUT: SMALL DATA**

25 rows is enough to learn one line, but it's tiny by machine-learning standards. Every result has to be read with that in mind: one unusual student can move the numbers. Be ready to say this in an interview before anyone asks.

REMEMBER

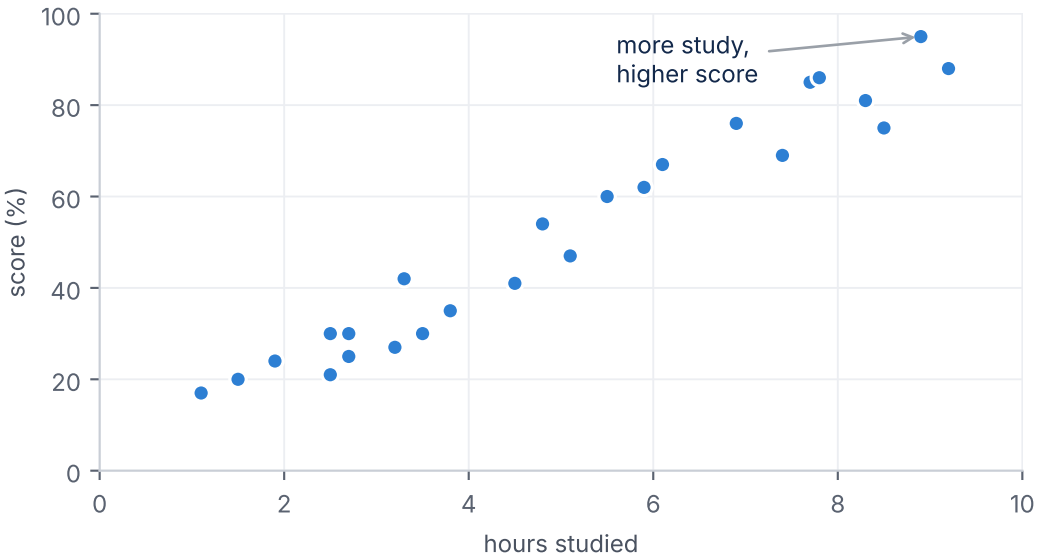
- Always look at the data first: `display()`, `describe()`, missing values, histograms.
- Our data: 25 students, hours 1.1 to 9.2, scores 17 to 95, nothing missing.

CHAPTER 4

Detective work

Before fitting a line, check that a line makes sense

Linear regression only works well if the relationship really is roughly a straight line. So the first step is a **scatter plot**: hours on the x-axis, score on the y-axis.



Each dot is a student. The dots rise from bottom-left to top-right in a fairly tight band: a clear straight-line pattern.

Putting a number on it: correlation

Correlation measures how strongly two things move together in a straight line, from −1 (opposite directions) through 0 (no straight-line link) to +1 (perfectly together).

0.976

correlation between hours and scores

That's extremely close to +1. Hours studied and scores move together almost perfectly, so a straight-line model is the right tool here.

🎤 **INTERVIEW TIP**

Correlation only catches **straight-line** relationships. Two things can be strongly linked in a curved way and still show a correlation near 0. That's why you always plot the data, not just compute the number.

⚠️ **WATCH OUT: CORRELATION IS NOT CAUSATION**

The data shows that students who study more *tend* to score higher. It doesn't prove studying *causes* the higher score: motivated students might both study more and do better for other reasons. Our model predicts; it doesn't explain why.

REMEMBER

- Plot first. A scatter plot tells you whether a line is the right shape.
- Correlation of 0.976 = a very strong, positive, straight-line relationship.

CHAPTER 5

The model

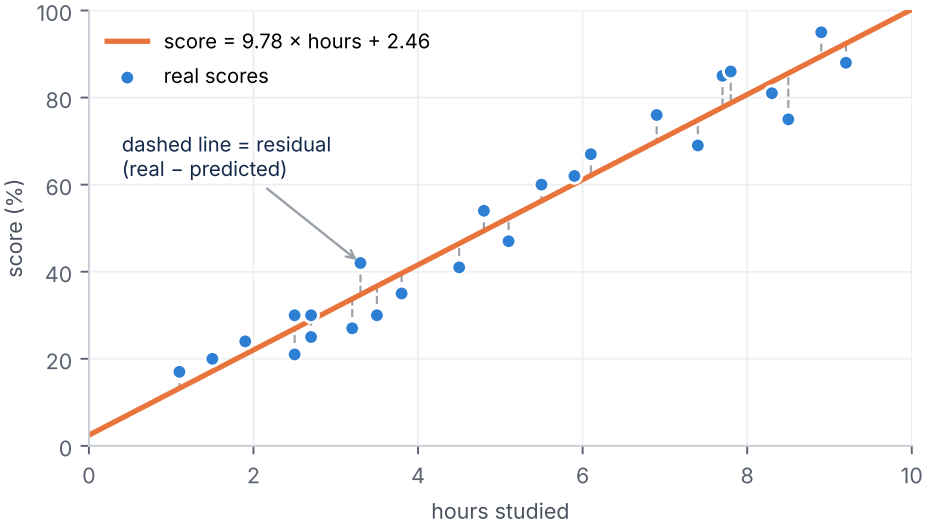
A straight line has only two numbers to learn

Every straight line can be written as:

$$\hat{y} = m \cdot x + b$$

Symbol	Name	What it means here	Our value
x	input	hours studied	given
ŷ ("y-hat")	prediction	predicted score	calculated
m	slope, weight or coefficient (B_1)	points gained for each extra hour	9.78
b	intercept or bias (B_0)	predicted score at 0 hours (where the line crosses the y-axis)	2.46

"Training" the model just means **finding the values of m and b that make the line fit the data best**, that is, with the smallest MSE. Here's the line the notebook found:



The orange line is the model. Each dashed grey line is a residual: how far a real score is from the line's prediction. Training makes these gaps, squared and averaged, as small as possible.

🗣️ **EXPLAIN IT TO AN AUDIENCE**

"Our model says: start at about 2 points, then add almost 10 points for every hour you study. So 5 hours gives about 2 + 49 = 51."

REMEMBER

- The model is a line: ŷ = m·x + b. Only two numbers to learn.
- m = 9.78 (points per hour), b = 2.46 (starting point).

CHAPTER 6

Gradient descent

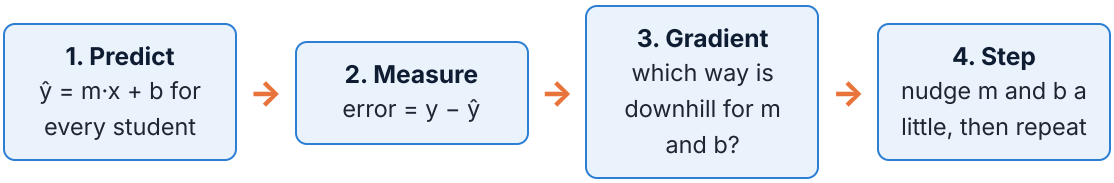
Walking downhill, one small step at a time

How do we find the best m and b? We could guess randomly forever. Instead, **gradient descent** starts with a guess (m = 0, b = 0) and improves it step by step, always moving in the direction that reduces the error.

🗣️ EXPLAIN IT TO AN AUDIENCE

You're on a foggy hillside and want to reach the bottom of the valley. You can't see far, but you can feel which way the ground slopes under your feet. So you take a small step downhill, feel again, and repeat. Eventually you reach the bottom. The **gradient** is the slope under your feet; the **valley floor** is the best m and b.

One iteration of the loop



The maths, gently

The error we minimise is the MSE. Calculus gives the slope of that error with respect to each number (these are exactly the lines in your notebook):

$$\partial \text{MSE}/\partial m = -(2/n) \cdot \Sigma\, x \cdot (y - \hat{y}) \qquad \partial \text{MSE}/\partial b = -(2/n) \cdot \Sigma\, (y - \hat{y})$$

Then each number takes a small step **against** its gradient (downhill), scaled by the learning rate L:

$$m \leftarrow m - L \cdot \partial \text{MSE}/\partial m \qquad b \leftarrow b - L \cdot \partial \text{MSE}/\partial b$$

🎯 WORKED EXAMPLE: THE VERY FIRST STEP

Start at m = 0, b = 0, so every prediction is 0 and every error is the real score.
b-gradient = −2 × (average score) = −2 × 51.48 = −102.96 → new b = 0 − 0.01 × (−102.96) = **1.03**
m-gradient = −2 × (average of hours × score) = −2 × 317.86 = −635.7 → new m = 0 − 0.01 × (−635.7) = **6.36**
One step and the line has already jumped from flat to steeply rising. The error drops from 3,264 to about 450.

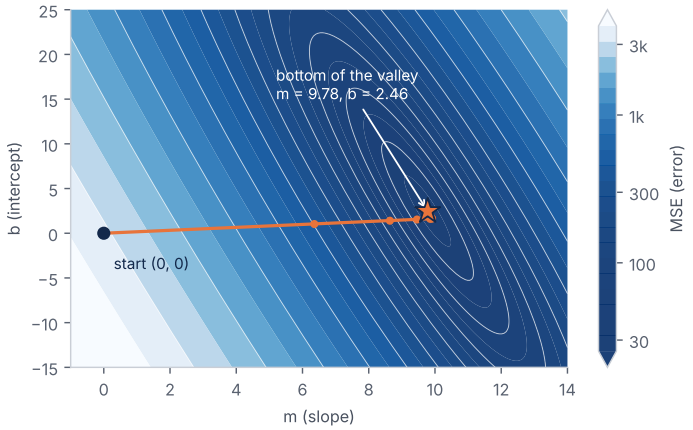
The code you wrote

```
def gradient_descent(m_now, b_now, points, L, iterations):
    n = len(points)
    for _ in range(iterations):
        m_gradient = 0
        b_gradient = 0
        for i in range(n):                                  # go through every student
            x = points.iloc[i].Hours
            y = points.iloc[i].Scores
            m_gradient += -(2/n) * x * (y - (m_now * x + b_now))
            b_gradient += -(2/n) * (y - (m_now * x + b_now))
        m_now -= m_gradient * L                             # step downhill
        b_now -= b_gradient * L
    return m_now, b_now

m, b = gradient_descent(0, 0, points, L=0.01, iterations=1000)   # → 9.7789, 2.4645
```

Seeing the valley

Every possible pair (m, b) has its own MSE. Colour them in and you get a map of the error "landscape". Darker means lower error. The orange path is gradient descent walking from the start to the bottom.



A long, narrow valley: many (m, b) pairs give similar errors, but only one is the lowest. A few big early steps, then many tiny ones.

🎤 **INTERVIEW TIP**

"Why not just use the exact formula?" For one feature you can, and I did, to check my answer. But gradient descent is what trains neural networks and scales to millions of parameters, where exact formulas get too slow. Writing it by hand shows I understand what happens inside `.fit()`.

⚠️ **WATCH OUT: SPEED**

The `.iloc[i]` loop handles one row at a time, which is slow on big data. The improved notebook adds a **vectorised** NumPy version that gives exactly the same m and b.

REMEMBER

- Gradient descent: predict → measure error → compute gradient → small step downhill → repeat.
- From (0, 0) it reached m = 9.78, b = 2.46 after 1,000 iterations.

CHAPTER 7

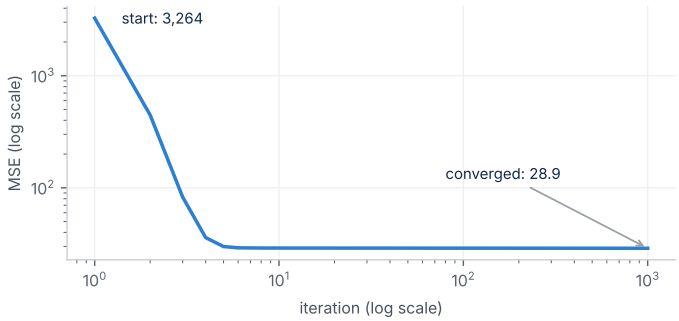
The learning rate

How big should each step be?

The **learning rate** (L) decides the size of each downhill step. It's a **hyperparameter**: a setting we choose before training, not something the model learns. We also chose the number of **iterations** (1,000).

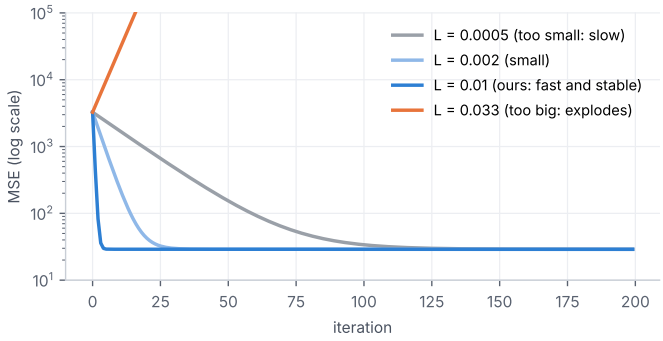
The loss curve: watching it learn

Plotting the MSE at every iteration shows whether training is working. Ours plunges in the first few steps, then flattens. Flat means the model has **converged**: further steps barely change anything.



Both axes use a log scale so the fast early drop is visible. The error goes from 3,264 to 28.9; most of the improvement happens in the first 5 iterations.

Too small, just right, too big



Small learning rates get there, but slowly. L = 0.01 converges in a handful of steps. L = 0.033 overshoots the valley on every step, so the error grows instead of shrinking.

🗣️ **EXPLAIN IT TO AN AUDIENCE**

Walking down into a valley: tiny shuffling steps take forever. Sensible strides get you there quickly. Giant leaps make you jump clean over the bottom and land higher up the other side, again and again.

🎤 **INTERVIEW TIP**

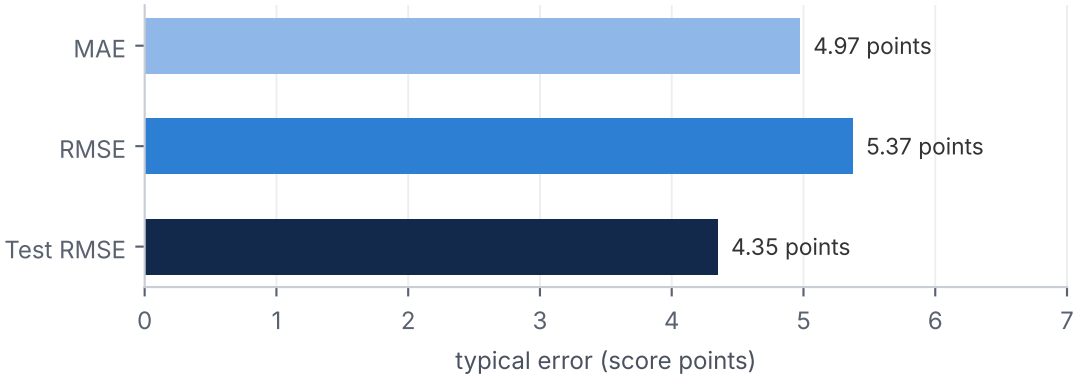
For this data, the largest learning rate that still converges is about **0.031** (it depends on the scale of the input). So 0.01 was a safe choice, and 0.033 just above the limit already blows up. That's also why **scaling features** matters in bigger projects: it makes one learning rate work well for every feature.

CHAPTER 8

How good is the model?

Putting numbers on the mistakes

Metric	Our value	How to say it
MSE	28.88	Average squared error (what gradient descent minimised)
RMSE	5.37 points	"Predictions are typically off by about 5 points"
MAE	4.97 points	"On average, the prediction misses by about 5 points"
CV of RMSE	10.4%	RMSE as a share of the average score (51.48). Errors are about a tenth of a typical score.
R^2	0.953	"Hours studied explain about 95% of the differences in scores"



All three error measures sit around 4 to 5.5 points on a 0 to 100 scale.

R^2: the "how much did we explain?" score

$$R^2 = 1 - (\text{sum of squared errors} \div \text{total variation around the mean})$$

Imagine a lazy model that always predicts the average score, 51.48. R^2 compares our line with that. **R^2 = 0** means we're no better than guessing the average; **R^2 = 1** means every prediction is perfect. Ours is 0.953.

💡 **IN PLAIN ENGLISH**

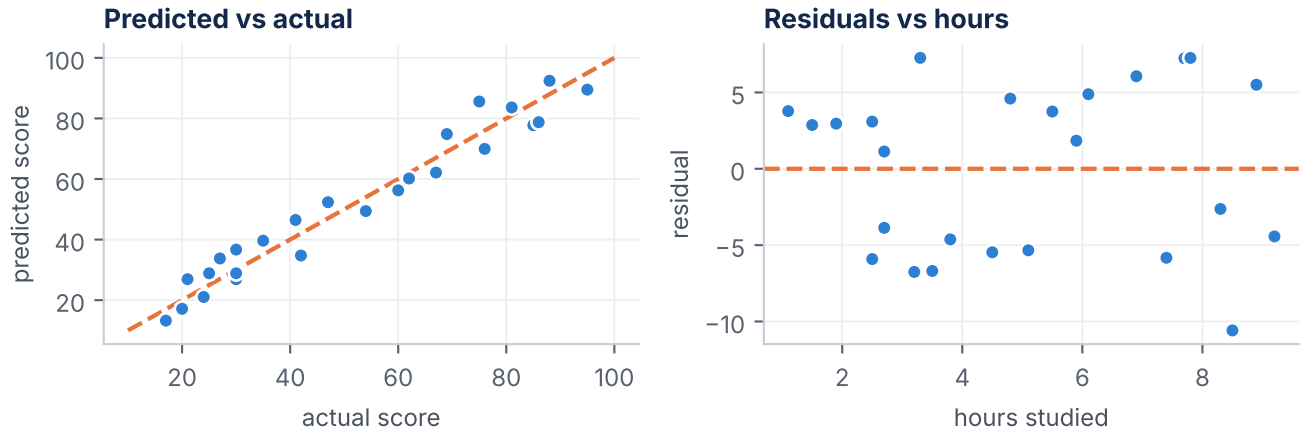
If you had to guess students' scores with no information, you'd be off by about 25 points (the standard deviation). Knowing their study hours cuts that to about 5 points.

🎤 **INTERVIEW TIP**

RMSE vs MAE? RMSE (5.37) is a bit higher than MAE (4.97) because squaring makes the bigger misses count more. When the two are close, as here, the errors are fairly even with no huge outliers. Report both: MAE is easier to explain; RMSE matches what the model optimised.

Looking at the mistakes: residual analysis

A good data scientist also asks **where** the model goes wrong. A residual is *real score − predicted score*. For a well-fitted line, residuals should look like random noise scattered around zero.



Left: predictions sit close to the dashed "perfect" line. Right: residuals bounce above and below zero with no clear curve or funnel shape.

What we see

- **Mean residual ≈ 0.004.** The model doesn't systematically over- or under-predict.
- **No pattern.** If residuals curved (negative in the middle, positive at both ends), a straight line would be the wrong shape.
- **Largest miss: 10.6 points.** A student who studied 8.5 hours scored 75; the model predicted about 86.

Patterns to watch for

If the residual plot shows…	It means…	Possible fix
A curve (U shape)	The relationship isn't a straight line	Add a squared term (polynomial regression)
A funnel (spread grows)	Errors get bigger for bigger x	Transform the target (e.g. log)
A few points far away	Outliers	Check them; consider MAE or robust regression
Random cloud around 0 ✓	The line is a good fit	Nothing: this is what we have

REMEMBER

- RMSE 5.37, MAE 4.97, R^2 0.953: a strong fit.
- Residual plots tell you whether the *shape* of the model is right, not just how big the errors are.

CHAPTER 9

Checking our answer

Did gradient descent find the right line? Does it work on new students?

Check 1: compare with the exact formula

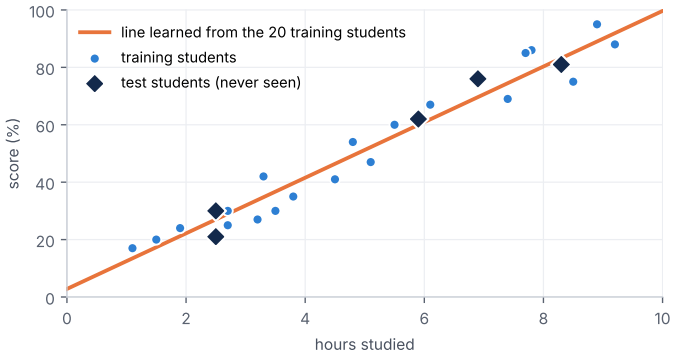
Simple linear regression has an exact solution called **ordinary least squares (OLS)**: $m = \Sigma(x - \bar{x})(y - \bar{y}) \div \Sigma(x - \bar{x})^2$, and $b = \bar{y} - m \cdot \bar{x}$. scikit-learn's `LinearRegression` uses this. If our hand-made gradient descent worked, it should land in the same place.

Method	m (slope)	b (intercept)	MSE
Our gradient descent (1,000 iterations)	9.7789	2.4645	28.8828
Exact formula (OLS)	9.7758	2.4837	28.8827
scikit-learn `LinearRegression`	9.7758	2.4837	28.8827

Practically identical. The tiny gap (0.02 in b) is because gradient descent is still creeping along the flat valley floor after 1,000 steps; more iterations would close it completely.

Check 2: a fair test on unseen students

So far we measured error on the same students the model learned from. That's like marking a student on the practice questions they already saw. A fairer test: **train on 20 students (80%) and test on the 5 the model has never seen (20%)**.



	RMSE	MAE	R^2
Training set (20 students)	5.61	5.22	0.949
Test set (5 unseen students)	4.35	3.92	0.968

The test error is similar to (even slightly lower than) the training error, so the model **generalises**: it isn't just memorising the students it saw.

> ⚠️ **WATCH OUT**
> With only 5 test students, the test score is noisy: a different random split could give a noticeably different number. On a small dataset, **k-fold cross-validation** (rotating which students are held out) gives a more reliable estimate.

CHAPTER 10

Predictions and limits

Using the model, and being honest about what it can't do

Making predictions

```
def predict_score(hours):
    return m * hours + b

predict_score(9.25)   # → 92.9
```

Hours studied	Calculation	Predicted score
2	9.78 × 2 + 2.46	22.0%
5	9.78 × 5 + 2.46	51.4%
9.25	9.78 × 9.25 + 2.46	92.9%

🗣️ **EXPLAIN IT TO AN AUDIENCE**

"A student who studies 9¼ hours can expect around 93%, give or take about 5 points."

Honest limitations of our model

Limitation	Why it matters	Possible fix
Only 25 students	Results can shift a lot with a few different students	Collect more data; use cross-validation
One feature	Sleep, attendance and prior grades also matter	Multiple linear regression with more features
Extrapolation	We only saw 1.1 to 9.2 hours. At 12 hours the line predicts 120%, which is impossible	Only predict inside the data range; cap at 100
Assumes a straight line	Real study likely has diminishing returns	Try a curve (polynomial) and compare
Correlation, not causation	Can't prove that studying more *causes* higher scores	A controlled experiment

⚠️ **WATCH OUT: EXTRAPOLATION**

A line goes on forever, but your data doesn't. The intercept (2.46 at 0 hours) is also an extrapolation, since nobody in the data studied less than 1.1 hours. Never trust a prediction far outside the range you trained on.

REMEMBER

- Prediction = m × hours + b. 9.25 hours → about 92.9%.
- Know the limits: small data, one feature, stay within 1 to 9 hours.

CHAPTER 11

Interview toolkit

Everything you need to talk about this project with confidence

Your 30-second pitch

🎤 SAY IT LIKE THIS

"I built a linear regression model from scratch to predict students' exam scores from the hours they studied. Instead of just calling scikit-learn, I implemented gradient descent myself, explored the data first (a correlation of 0.98), and tracked the loss curve and the effect of the learning rate. The model learned that each extra hour adds about 9.8 points, with a typical error of about 5 points and an R^2 of 0.95. I verified it against the exact least-squares solution and scikit-learn, and tested it on held-out students. Its main limitations are the small dataset and the single feature."

Questions you might be asked

Q. Walk me through your project.

Frame the problem → load and explore the data → check a line fits (scatter, correlation) → define the model $\hat{y} = mx + b$ → train it with gradient descent → evaluate with MSE, RMSE, MAE, R^2 and residuals → verify against OLS and a train/test split → make predictions.

Q. What is gradient descent?

An optimisation method. Start with a guess for the parameters, compute the gradient (the direction the error increases fastest), take a small step the opposite way, and repeat until the error stops improving.

Q. How did you choose the learning rate and number of iterations?

I tried several values and watched the loss curve. 0.01 converged within a few iterations; larger values like 0.033 diverged. 1,000 iterations was more than enough, since the curve was flat long before that.

Q. How do you know the model converged?

The loss curve flattened, and the final m and b matched the exact OLS solution to within about 0.02.

Q. Why MSE as the loss function?

It's always positive, punishes large errors more, and is smooth and bowl-shaped for linear regression, so there's a single minimum and gradient descent can't get stuck.

Q. What does $R^2 = 0.95$ mean?

The model explains about 95% of the variation in scores compared with always guessing the average.

Q. What are the assumptions of linear regression?

A linear relationship; independent observations; residuals with constant spread (homoscedasticity); and, for statistical inference, roughly normal residuals. My residual plot showed no pattern, which supports the first and third.

Q. Is the bowl always one valley? Could gradient descent get stuck?
For linear regression with MSE the error surface is **convex**: one global minimum, no local traps. That's not true for neural networks, where gradient descent can land in a local minimum.

Q. Batch, stochastic or mini-batch gradient descent?
I used **batch** gradient descent: every step uses all 25 students. Stochastic uses one row per step (fast, noisy); mini-batch uses small groups (the usual choice for big data).

Q. What would you do with more time?
Collect more data, use k-fold cross-validation, add features such as sleep or attendance (multiple regression), try a polynomial term to test for diminishing returns, and add a stopping rule that ends training when the loss stops improving.

Q. What's the difference between overfitting and underfitting?
Underfitting: too simple, bad on both training and new data. Overfitting: memorises training data and does badly on new data. My train and test errors were similar (5.6 vs 4.4), so neither is a problem here.

Q. What is the intercept, and does it make sense?
The predicted score at 0 hours: 2.46. It's mathematically needed to position the line, but it's an extrapolation, since nobody in the data studied under 1.1 hours.

Glossary

Term	Meaning
Feature / label	The input (hours) / the answer to predict (score)
Slope (m) / intercept (b)	The line's steepness / where it crosses the y-axis. Also called coefficients, weights and bias.
Loss function	The number training tries to make small (here MSE)
Gradient	The slope of the loss: which way, and how steeply, it increases
Learning rate	Step size in gradient descent
Iteration / epoch	One update of m and b. With batch gradient descent, one iteration = one pass through all the data (an epoch).
Convergence	When further steps no longer reduce the loss meaningfully
Hyperparameter	A setting chosen before training (learning rate, iterations)
Residual	Real value − predicted value
R^2	Share of the variation in the target that the model explains
OLS	Ordinary least squares: the exact formula for the best line
Train/test split	Holding back some data to test the model on examples it hasn't seen
Vectorisation	Doing maths on whole arrays at once (NumPy) instead of a Python loop

ONE-PAGE REVIEW

Cheat sheet

Read this five minutes before the interview

THE PROBLEM

Predict exam score (0 to 100) from hours studied. Supervised, simple linear regression. 25 students, no missing values.

THE MODEL

$$\text{score} = 9.78 \times \text{hours} + 2.46$$

Each extra hour ≈ +9.8 points.

TRAINING

Gradient descent from (0, 0), learning rate 0.01, 1,000 iterations. MSE fell from 3,264 to 28.9. Converged in about 5 steps. Diverges above L ≈ 0.031.

VERIFICATION

Matches the exact OLS formula and scikit-learn (m 9.776, b 2.484). Train RMSE 5.61, test RMSE 4.35: generalises well.

Results

0.976	5.37	4.97	0.953	10.4%
correlation	RMSE (points)	MAE (points)	R^2	CV of RMSE

The formulas

Prediction	$\hat{y} = m \cdot x + b$
MSE	$(1/n)\ \Sigma\ (y - \hat{y})^2$
Gradients	$\partial/\partial m = -(2/n)\ \Sigma\ x(y - \hat{y})$ $\quad \partial/\partial b = -(2/n)\ \Sigma\ (y - \hat{y})$
Update	$m \leftarrow m - L \cdot \partial/\partial m$ $\quad b \leftarrow b - L \cdot \partial/\partial b$
RMSE / MAE	$\sqrt{\text{MSE}}$ $\quad / \quad$ $(1/n)\ \Sigma\ \lvert y - \hat{y} \rvert$
R^2	$1 - \Sigma(y - \hat{y})^2 \div \Sigma(y - \bar{y})^2$
OLS slope	$m = \Sigma(x - \bar{x})(y - \bar{y}) \div \Sigma(x - \bar{x})^2$, $\quad b = \bar{y} - m \cdot \bar{x}$

Three things to always mention

1. **I built gradient descent myself**, then verified it against the exact answer.
2. **I checked the fit visually:** scatter plot, loss curve, residual plot.
3. **I know the limits:** small data, one feature, no extrapolation, correlation ≠ causation.